

1. RAW - Read After Write

ADD R1, R2, R3 write to R1 (in WB stage)

SUB R4, R1, R5 reads R1 (in ID stage)

Cycle:	1	2	3	4	5	
ADD R1:	IF	ID	EX	MEM	WB	R1 written here (cycle 5)
SUB R4:		IF	ID	EX	MEM	R1 read here (cycle 3)

Solutions:

1. Stalling - insert NOPs to wait for WB to complete.
2. Forwarding - route the result from EX or MEM stage directly back to where it's needed, without waiting for WB - using the pipeline registers

Things needed to be verified:

1. When instruction N writes to register Rx, and instruction N+1 or N+2 reads from Rx, the correct (updated) value must be used.
2. If forwarding is implemented, verify the forwarded value is correct.
3. If stalling is implemented, verify the correct number of stall cycles were inserted

2. WAR - Write After Read

SUB R4, R1, R5 reads R1 (in ID stage)

ADD R1, R2, R3 writes to R1 (in EX/WB stage)

If ADD writes to R1 before SUB has finished reading R1, SUB gets the wrong (newer) value.

Cycle:	1	2	3	4	5	
SUB R4:	IF	ID	EX	MEM	WB	R1 read in cycle 2 (ID)
ADD R1:		IF	ID	EX	MEM	R1 written in cycle 5 (WB)

In a standard 5-stage in-order MIPS pipeline, WAR hazards rarely cause actual problems because the ID of the earlier instruction always happens before the WB stage of the later instruction.

1. The earlier instruction reads the old value of the register, not the value written by the later instruction
2. Check for out of order feature

3. WAW - Write After Write

ADD R1, R2, R3 writes to R1

MUL R1, R4, R5 also writes to R1

After both complete, R1 should hold the result of MUL

Cycle:	1	2	3	4	5	6	7	
ADD R1:	IF	ID	EX	MEM	WB			writes R1 at cycle 5
MUL R1:		IF	ID	EX	EX	EX	WB	multi-cycle, writes R1 at cycle 7

Here MUL finishes last, so R1 = MUL result. If variable-latency instructions, the order can flip. Check multi-cycle instructions like MUL(shift and add), DIV(repeated sub)

RAW sequences

1. 1 instruction gap between writer and reader
2. 2-instruction gap
3. 3+ instruction gap (should be safe)

WAR sequences

1. Two writes to the same register back to back
2. Two writes with one instruction in between
3. A multi-cycle instruction (MUL/DIV) followed by a regular write

WAW sequences

1. A read followed immediately by a write to the same register
2. Verify the read captured the old value

Do we need to have a golden reference model like with the help of processor outputs using the scoreboard we see what and how many mismatches ?

RAW coverage:

- RAW with 0 instructions between writer and reader
- RAW with 1 instruction between them
- RAW with 2 instructions between them
- RAW involving different instruction types (ADD,SUB, LW,ADD, etc.)
- RAW with forwarding path exercised

Flown of the code ?

Signals to be seen on the DUT at the WB stage:

1. **wb_reg_write** - whether a register is being written in the current cycle (1 or 0).
2. **wb_reg_dest** - specifies which register is being written
3. **wb_reg_data** - value written to that register.

Every clock cycle, we check the following condition:

If `wb_reg_write = 1` then `register[wb_reg_dest] = wb_reg_data`.

and send it to the scoreboard so it can be compared with the expected value.

We also need to check some additional signals

- **stall signal** - Indicates whether the processor inserted a stall cycle to resolve a RAW hazard.
- **forward_flag** - To check if the forwarding unit was activated.
- **EX stage operand values** – Confirms whether the instruction received the correct input operands.